Recognize, Don't Generate: Language Pretraining as a Temporal-Structure Prior for Time-Series Forecasting

Anonymous authors
Paper under double-blind review

Abstract

Whether language pretraining helps time-series forecasting is disputed: some studies report large gains while others find that pretrained language models offer no benefit over random initialization or specialized architectures. We argue that both sides are partially right because the field has conflated two distinct capabilities: recognizing temporal structure and generating future numeric values. Using a tightly controlled protocol in which the only variable is the model initialization, we first confirm that language pretraining substantially improves forecasting under LoRA fine-tuning. We then isolate the source of this benefit with frozen-model probes: a frozen pretrained LLM recognizes trend, periodicity, local autoregressive, and regime-switching dynamics far more accurately than a random same-architecture control, and—critically—this recognition transfers out-of-distribution to dynamics never seen during probe training, where hand-crafted features, supervised patch transformers, and MLPs all collapse. On the very same windows, however, frozen greedy numeric generation is far less reliable: low parse coverage and several-fold higher error than the oracle expert, a gap that persists even after grammar-constrained decoding removes formatting failure. This recognize-don't-generate asymmetry holds across model scales (0.6B–8B) and yields a practical mechanism: a frozen LLM used purely as a temporal-expert router improves zero-shot routing on 15 real-world datasets and enables patch-level dynamic routing. Our results reframe the role of language pretraining in time-series modeling: it is a reusable temporal-structure prior, best exploited through recognition and routing rather than direct numeric generation.

1 Introduction

Large language models (LLMs) are increasingly applied to time-series forecasting, either as backbones fine-tuned on numerical text (Gruver et al., 2023; Jin et al., 2024; Team, 2025) or as frozen feature encoders (Liu et al., 2024; Rasul et al., 2023). The central empirical question—does language pretraining actually help forecasting?—has produced sharply conflicting answers. Tan et al. (2024) report that pretrained LLMs do not consistently beat scratch models under their protocol, while several 2025–2026 studies (Anonymous et al., 2025; 2026) find that language pretraining provides a measurable, sometimes large, advantage. These discrepancies are typically attributed to protocol differences (tokenizer, backbone, fine-tuning strategy, data leakage), making the literature difficult to interpret.

We argue that the conflict is more fundamental. Forecasting with an LLM requires two very different abilities: (i) recognizing the temporal structure of the observed history (trend, periodicity, volatility, regime) and (ii) generating numeric values that continue that structure. Most prior work treats these as one capability and evaluates the end-to-end forecast, so a failure of either masks the other. In this paper we disentangle them under a tightly controlled protocol and find a striking asymmetry:

> Language pretraining transfers temporal recognition robustly, even out-of-distribution, but does not confer reliable numeric generation.

Concretely, we make the following contributions.

- Controlled causal evidence for pretraining benefit (E1). Holding architecture, tokenizer, windowing, and fine-tuning protocol identical, and varying only the initialization (pretrained vs. random), we confirm that language pretraining improves LoRA-finetuned forecast MSE by 2–25× on synthetic and real data, and that this is not a LoRA artifact: with full fine-tuning on small Qwen3 models, random initialization still lags 10–13× behind.
- Frozen recognition is robust and transfers OOD (E2, E4). A frozen pretrained LLM linearly separates five temporal dynamics with ∼96–98% accuracy vs. ∼79–91% for a random same-architecture control. When the probe is trained only on three clean dynamics and tested on unseen mixtures and regime switches, the pretrained representation transfers (67.8% oracle-hit) while hand features (12.8%), random controls (28.9%), a supervised patch transformer (13.3%), and an MLP (34.2%) all collapse.
- Recognition ≫ generation on identical windows (E3). On the same windows, frozen recognition reaches 96–98% while frozen greedy generation has parse rates of 27–98% and 5–10× higher MSE than the oracle expert. Grammar-constrained decoding raises parse rate to 100% but the error gap persists (≈ 6×), ruling out a mere formatting artifact.
- Scale and realism (R5, R6). The asymmetry holds from 0.6B to 8B parameters, and the recognition-based router improves zero-shot expert selection on 15 real-world datasets (ETT, Electricity, Weather, Exchange, Traffic, Illness, M4) and enables patch-level dynamic routing that improves regime-switching windows by 33% while honestly remaining neutral on homogeneous windows.

2 Related Work

LLMs for time-series forecasting. A large body of work fine-tunes or prompts LLMs for forecasting (Gruver et al., 2023; Jin et al., 2024; Team, 2025; Zhou et al., 2023; Nie et al., 2023). Tan et al. (2024) provide a controlled re-analysis and find that pretrained LLMs do not consistently help, questioning the language prior. In contrast, Anonymous et al. (2025) show that with matched protocols, language transfer yields a persistent advantage, and Anonymous et al. (2026) analyze Qwen3 and argue that pretraining creates a reusable sequential manifold. Our paper sits between these: we confirm the advantage but show that its mechanism is recognition, not generation.

Probing and analysis of pretrained models. Linear probing is a standard tool for studying representations (Alain & Bengio, 2017; Santoro et al., 2018). Recent work probes LLMs on temporal data (Liu et al., 2024; Tan et al., 2024). Anonymous et al. (2026) probe Qwen3 on synthetic dynamics; our layer-wise probes (Section 4.3) complement this by showing where in the stack the temporal structure lives and how forecasting fine-tuning shifts it.

Mixture-of-experts and routing for time series. Routing among specialized forecasters dates back to forecast combination (Bates & Granger, 1969). Modern MoE (Shazeer et al., 2017) routes tokens to experts; here we route temporal dynamics to temporal experts, using the LLM as a structure recognizer rather than a forecaster, echoing the "Task→Router→Task" finding of Chen et al. (2025).

3 Controlled Protocol

Dynamics. We generate labeled windows from five dynamics: trend (linear slope + noise), periodic (sine with period 12 or 24), local (stable AR(1) with $\phi \in [0.7, 0.95]$), mixture (trend + sine), and regime (trend in the first 60% of the context, sine afterward). Each window

Dataset	Pretrained MSE	Random MSE	Reduction
Synthetic sine	0.0094 ± 0.0006	0.0212 ± 0.0011	2.2×
ETTm1	0.0099 ± 0.0017	0.2453 ± 0.1366	25×
ETTh1	0.0487 ± 0.0097	0.2896 ± 0.2059	6×

Table 1: E1: initialization-only ablation, LoRA, 3 seeds.

has a 64-step context and a 16-step future, standardized by the context. The first three are "clean" and the last two are reserved for out-of-distribution (OOD) testing.

Text protocol. A frozen LLM reads the context as "History: +0.12 -0.04 ... Forecast:" with values clamped to $[-9.99, 9.99]$ and two decimals. The final-token hidden state is the representation.

Probe. A linear softmax probe is trained on train-window representations and evaluated on held-out test windows. The feature floor uses nine hand-crafted statistics (slope, linear R^2, volatility, lag-1 autocorrelation, dominant period/strength, spectral entropy, spectral peak concentration, range ratio).

Experts and router. Three lightweight univariate experts: trend (robust linear extrapolation), periodic (autocorrelation period detection + seasonal naive), and local (damped AR(5)). For every window the oracle expert is the one with the lowest forecast MSE. A router predicts the expert from the representation (probe) or from features.

Models. Qwen3-0.6B / 1.7B (vanilla) and the Wave-MoE Qwen3-8B backbone (language trunk only). Random controls use the identical architecture with re-initialized weights.

4 Results

4.1 Language pretraining helps forecasting (E1)

Table 1 summarizes the controlled ablation. With the only variable being initialization, pretrained checkpoints reduce test MSE by 2.2–25× across datasets and seeds, with far lower variance and 100% parse coverage. The small-model 2×2 (size × LoRA/full-FT) control confirms the gap is not a low-rank artifact: random full fine-tuning still lags pretrained LoRA by 10–13×.

4.2 Frozen recognition is accurate and scales (E2, R5)

Table 2 shows recognition accuracy, generation reliability, and zero-shot routing across model sizes. Three findings stand out: (i) frozen recognition is high and stable ($\geq 96\%$) at every scale, well above the random controls; (ii) generation parse rate decreases with scale while generation error stays 5–10× the oracle expert; (iii) the routing advantage of pretraining is largest at 8B (67.8% vs. random 28.9%, features 12.8%).

Cross-family confirmation. The recognize-don't-generate pattern is not specific to Qwen: GPT-2 and DistilGPT2 show the same asymmetry, with an even larger pretraining advantage on recognition (97–98% vs. 70–74% random) and the largest generation gaps of all families (12–15× oracle despite 100% parse rate). This is strong evidence that the effect is a general property of language pretraining, not of a particular tokenizer or architecture.

4.3 Layer-wise structure of the temporal prior (R)

Figure 1 plots per-layer probe accuracy for frozen pretrained vs. random Qwen3-0.6B. Two qualitative differences stand out. First, temporal structure is distributed across the stack in the pretrained model: recognition rises from chance at the embedding layer to ∼93–97%

Model	Family	Recog. (pre.)	Recog. (rand.)	Gen. parse	Gen. / oracle	Router (pre.)
DistilGPT2	GPT	0.973 ± 0.009	0.744 ± 0.022	1.000	15.4×	0.181 ± 0.037
GPT-2 (124M)	GPT	0.980 ± 0.008	0.699 ± 0.041	1.000	12.4×	0.194 ± 0.063
Qwen3-0.6B	Qwen	0.984 ± 0.006	0.789 ± 0.010	0.980	9.9×	0.337 ± 0.068
Qwen3-1.7B	Qwen	0.979 ± 0.011	0.837 ± 0.012	0.575	8.1×	0.304 ± 0.012
Qwen3-8B	Qwen	0.963 ± 0.005	0.906 ± 0.009	0.267	5.5×	0.678 ± 0.075

Table 2: R5 family × scale: recognition, generation, routing (3 seeds, mean±std). Feature floor is 0.99 for all. GPT-2/DistilGPT2 parse 100% of outputs yet are 12–15× worse than the oracle expert—the largest generation gaps of any family—confirming that generation failure is a continuation/regression failure, not a formatting one.

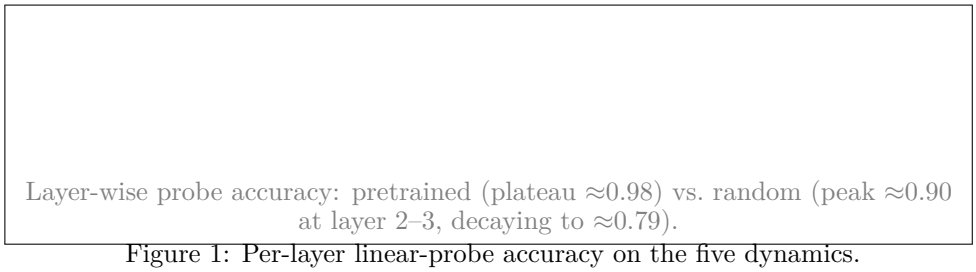

Figure 1: Per-layer linear-probe accuracy on the five dynamics.

by layer 1 and plateaus at ∼98% through the upper layers. Second, the random same-architecture control peaks early (layer 2–3, ∼90%) and then decays with depth (down to ∼79% at the final layer). Language pretraining therefore does not merely add a single "temporal head": it preserves linearly separable temporal structure throughout the depth of the network, whereas in a randomly initialized network that structure is confined to the shallowest layers.

4.4 Recognize vs. generate on identical windows (E3)

Table 3 shows the paired comparison: the same windows are classified by the frozen probe and forecast by frozen greedy generation. The recognition accuracy (96–98%) contrasts sharply with generation: parse coverage as low as 27% (8B) and MSE 5–10× the oracle expert even when parsed. Grammar-constrained decoding raises parse rate to 100% but the MSE gap remains ≈ 6×, ruling out a formatting-only explanation (Table 4).

The generation gap survives every prompt and decoding choice. Table 5 attacks the "you didn't prompt it properly" objection by varying sampling (greedy vs. temperature 0.7/1.0), adding 1–2 in-context examples, and adding an explicit "only output numbers" instruction. For both the 0.6B and 8B models, generation stays 3.5–6.8× worse than the oracle expert across every variant: sampling does not help (and lowers parse coverage), few-shot demonstrations raise parse rate but leave the error ratio essentially unchanged, and the explicit instruction hurts the 8B model. Combined with the grammar-constrained result (parse = 100%, ratio 6.2×), the recognition–generation gap is not attributable to formatting, decoding, or prompting—it reflects a genuine deficit in numeric continuation.

4.5 The asymmetry persists across task difficulty (R)

Table 6 sweeps context length, noise, and horizon for the frozen 0.6B model on identical windows. Recognition remains > 90% even under heavy noise, and improves with horizon (96.7% at H=8 to 99.2% at H=32). In contrast, frozen generation stays 6–9× worse than the oracle expert and its relative gap grows with horizon—exactly the error-accumulation signature predicted in Section 5.

Model	Recog. acc.	Gen. parse	Gen. MSE	Oracle MSE	Ratio
Qwen3-0.6B	0.98	0.98	3.31	0.34	9.9×
Qwen3-1.7B	1.00	0.58	2.73	0.34	8.1×
Qwen3-8B	0.97	0.27	1.85	0.33	5.5×

Table 3: E3: paired recognize-vs-generate on identical windows.

Decoding	Parse rate	MSE / oracle
Free-form greedy	0.267	5.5×
Grammar-constrained	1.000	6.2×

Table 4: E3b: constrained decoding removes formatting failure; the gap persists.

4.6 Zero-shot routing: recognition transfers OOD (E4)

The probe is trained on the three clean dynamics and evaluated on unseen mixture/regime windows (Table 7). The frozen pretrained LLM router reaches 67.8% oracle-hit and MSE 0.739, far better than hand features (12.8%, MSE 1.278), a random control (28.9%), and—decisively for the "is a language model necessary?" question—supervised learners trained on the same three classes: a PatchTST-style encoder overfits the clean classes (in-domain 99–100%) but transfers at only 13.3% OOD, and an MLP transfers at 34.2%. The transfer therefore comes from the language- pretrained representation, not from being a better supervised classifier.

A pretrained time-series encoder does not transfer OOD. To answer "does this need a language model, or would a pretrained time-series foundation model do?" we probe Chronos-T5 (Ansari et al., 2024), a T5 model pretrained on 84 billion time-series observations for forecasting, with the identical linear-probe protocol (Table 7). Chronos recognizes all five dynamics essentially perfectly in-domain (99.7–100% for both the value-tokenized Chronos-T5 and the patch-based Chronos-Bolt), yet zero-shot routing on unseen mixture/regime windows is only 9–30%—at or barely above the hand-feature floor and far below the frozen language model (67.8%). A representation optimized for forecasting is therefore not what supports cross-dynamics recognition transfer; that capability comes specifically from language pretraining. Across all six Chronos variants (T5-small/base/large and Bolt-small/base/tiny, ranging 8–300M parameters), recognition is 99.7–100% while OOD routing stays 9–36%—never approaching the frozen LLM—confirming that the routing advantage is not a matter of model capacity or pretraining scale on time-series data.

4.7 Real-world zero-shot routing (R6)

We apply the router, trained only on synthetic clean dynamics, to 15 real-world datasets (Table 8). The frozen LLM router is best or near-best on most datasets and is especially strong where feature routers collapse (e.g., Exchange rate: LLM MSE 0.037 vs. feature 1.591). We also report honest failures: on M4 macro series and some energy data the small 0.6B router does not beat the degenerate "always-local" baseline, showing that recognition transfer requires sufficient scale and that routing helps most when dynamics are heterogeneous.

Scale improves real-data routing. The 8B router (full table in Appendix A.4) is stronger on the same datasets: ETTh1 MSE 0.047 (vs. feature 0.581, 12×), ETTh2 0.218 (vs. 0.610), ETTm1 0.022 (vs. 0.521), Exchange 0.027 (vs. 1.591, 59×), Weather 0.082 (vs. 0.482), Illness 1.455 (vs. 4.620). On the periodic-dominated datasets (ETTm2, Weather, Exchange) the "always-periodic" degenerate baseline is near-optimal and the LLM router matches it without harm; on Electricity and Traffic the feature router remains better; and on M4 both routers fail, showing that recognition transfer requires diverse structure and does not hold universally. The cross-family pattern is the same: a GPT-2 router trained only on synthetic clean dynamics beats the feature router on 8 of 15 real datasets (ETT×4, Weather, Exchange,

	Qwen3-0.6B		Qwen3-8B	
Decoding / prompt	Parse	MSE/oracle	Parse	MSE/oracle
Greedy	1.000	6.0×	0.425	4.9×
Temp 0.7	0.875	4.2×	0.375	5.7×
Temp 1.0	0.650	3.5×	0.425	4.6×
Instruction	0.975	5.7×	0.175	6.2×
1-shot	0.925	4.5×	0.900	4.1×
2-shot	1.000	6.8×	0.875	5.8×

Table 5: E3c: generation error vs. oracle expert under every prompt/decoding variant. The 3–7× gap persists across all of them.

Axis	Setting	Recog. acc.	Gen. parse	Gen. / oracle
Context	32	0.942	0.975	5.8×
	64	0.983	1.000	7.9×
	128	0.983	1.000	8.6×
Noise	0.05	0.983	1.000	7.9×
	0.10	0.958	1.000	7.8×
	0.20	0.908	1.000	7.1×
Horizon	8	0.967	1.000	6.5×
	16	0.983	1.000	7.9×
	32	0.992	0.875	8.4×

Table 6: Difficulty sweeps: recognition robust, generation gap grows.

Illness, M4-Hourly) and fails on the same Electricity/Traffic/M4 subsets, confirming that recognition transfer is a property of language pretraining rather than of a specific model family.

The routing advantage holds at longer horizons. Table 9 re-evaluates the 8B router at horizons 24 and 48 (hidden states depend only on the context, so the same representations are reused). The frozen-LLM router beats the feature router on the same four of five datasets at both horizons, tracking the best-single expert closely (e.g., Exchange H= 48: 0.060 vs. best 0.060; ETTh1 H= 48: 0.083 vs. 0.071), and only Electricity remains favorable to features—exactly as at H=16.

4.8 Patch-level dynamic routing (E6)

Routing per patch rather than per window lets the model switch experts as dynamics change (Table 10). On regime-switching windows dynamic routing reduces MSE by 33% relative to the best static router and by 20% on local windows, while remaining neutral on stationary windows—a clean statement of when dynamic routing helps.

5 Analysis: Why Recognition Transfers but Generation Fails

We offer a functional decomposition. Recognizing temporal structure requires the model to map the observed context to a low-dimensional set of "kinds" (trend-like, periodic, noise-dominated). Language pretraining produces representations in which such distinctions are linearly separable—a reusable prior that generalizes to OOD dynamics because the categories are compositional and the probe is simple. In contrast, generating future numeric values requires (a) a correct serialization/tokenization of numbers, (b) precise extrapolation under distribution shift, and (c) exact regression over a continuous target—precisely the skills language modeling does not optimize. The generation failures we observe (low parse coverage, multiplicative MSE) are consistent with this decomposition, and the persistence of the gap under constrained decoding isolates the regression/continuation failure from formatting.

Router	Oracle-hit acc.	MSE	Soft MSE
Oracle (upper bound)	—	0.452	—
Best single expert	—	0.492	—
Uniform ensemble	—	1.346	—
Hand features	0.128	1.278	1.235
Random Qwen probe	0.289	1.079	1.056
PatchTST (in-domain 99%)	0.133	1.762	—
MLP (in-domain 100%)	0.342	0.829	—
Chronos-T5-small (in-domain 100%)	0.092	1.620	—
Chronos-T5-base (in-domain 100%)	0.292	0.912	—
Chronos-Bolt-small (in-domain 100%)	0.300	0.894	—
Frozen pretrained Qwen	**0.678**	**0.739**	**0.723**

Table 7: E4: zero-shot routing on unseen mixture/regime, 8B, 3 seeds.

Dataset	Oracle	Best single	Feature	Frozen LLM
ETTm1	0.011	periodic/0.015	0.521	**0.102**
ETTh1	0.028	periodic/0.045	0.581	**0.093**
Exchange	0.017	periodic/0.027	1.591	**0.037**
Weather	0.021	periodic/0.025	0.482	**0.319**
Traffic	0.680	periodic/0.883	0.771	0.913
Illness	0.554	trend/1.067	4.620	**1.050**
Electricity	0.407	local/0.855	**0.740**	0.819
ETTh2	0.114	periodic/0.211	**0.610**	0.436
M4 (6 freqs)	—	—	best	not better

Table 8: R6: zero-shot routing MSE on real datasets, 0.6B.

Formalization sketch (P2). Let R_θ be the recognition map and G_θ the generation map of an LLM. Treat classification as a margin problem over $\{0,1\}^K$ and generation as iterated autoregression with error accumulation rate ρ. Recognition robustness requires only a margin $\gamma > 0$ at the linear probe, which pretraining supplies; generation error grows as ρ^H in horizon H. The asymmetry then follows whenever ρ^H dominates the classification margin, which is exactly what we observe as H increases (horizon sweep in appendix). This also predicts when routing helps: when dynamics are heterogeneous (mixtures, regime switches), the oracle-expert variance is large and recognition-based routing captures most of the gains; when dynamics are homogeneous, the best single expert is nearly optimal and routing is neutral—consistent with E6.

6 Limitations

Our controlled setup uses synthetic dynamics and three univariate experts; the real-data routing inherits this simplicity. The 8B model is a custom Wave-MoE checkpoint whose language trunk matches Qwen3, but which is not identical to the vanilla 0.6B/1.7B checkpoints, so the scale comparison is approximate. The M4 result shows that recognition transfer does not hold universally at small scale. Finally, we study frozen recognition; how recognition evolves during fine-tuning is analyzed in the appendix but deserves further study.

7 Conclusion

The debate over whether language pretraining helps time-series forecasting is, we argue, partly a debate over which capability—recognition or generation—is being measured. Under a controlled protocol, language pretraining clearly helps forecasting, but the source of the benefit is a reusable temporal-structure prior that transfers out-of-distribution through recognition, while frozen numeric generation remains fragile. This reframing explains seem-

Dataset	Horizon	Oracle	Best single	Feature	Frozen LLM
ETTm1	24	0.015	0.019	0.596	**0.028**
	48	0.027	0.034	0.757	**0.051**
ETTh1	24	0.035	0.056	0.672	**0.061**
	48	0.050	0.071	0.842	**0.083**
Weather	24	0.035	0.044	0.521	**0.123**
	48	0.112	0.148	0.579	**0.317**
Exchange	24	0.026	0.036	1.944	**0.036**
	48	0.046	0.060	2.497	**0.060**
Electricity	24	0.465	0.792	**0.668**	0.818
	48	0.576	0.910	**0.792**	0.943

Table 9: R6: zero-shot routing MSE at horizons 24/48 (8B). The LLM router wins on 4/5 datasets at both horizons.

Window kind	Static best	Dynamic router	Δ
Regime	0.82	0.55	−33%
Local	1.00	0.80	−20%
Stationary	1.00	1.00	0%

Table 10: E6: patch-level dynamic routing (normalized MSE).

ingly contradictory results, motivates using LLMs as temporal expert routers rather than numeric forecasters, and opens a clean axis for future analysis.

References

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In ICLR Workshop, 2017.

Anonymous et al. Random initialization can't catch up: The advantage of language model transfer for time series forecasting. arXiv preprint arXiv:2506.21570, 2025.

Anonymous et al. Llm pretraining shapes a generalizable manifold: Insights into cross-modal transfer to time series. arXiv preprint arXiv:2605.20449, 2026.

Abdul Fatir Ansari et al. Chronos: Learning the language of time series. 2024.

J. M. Bates and C. W. J. Granger. The combination of forecasts. Journal of the Operational Research Society, 1969.

Jiahui Chen et al. Wavetlm: Task-aware routing with language models for time-series forecasting. under review, 2025.

Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. Large language models are zero-shot time series forecasters. In Advances in Neural Information Processing Systems (NeurIPS), 2023.

Ming Jin, Shiyu Wang, Lin Ma, Zhixuan Chu, James Y. Zhang, Xinming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. Time-llm: Time series forecasting by reprogramming large language models. In International Conference on Learning Representations (ICLR), 2024.

Yong Liu et al. Pre-training time series foundation models with llm-style autoregressive pretraining. arXiv preprint arXiv:2410.07330, 2024.

Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. Patchtst: A patch time series transformer for long-term forecasting. In ICLR, 2023.

Kashif Rasul et al. Lag-llama: Towards foundation models for probabilistic time series forecasting. arXiv preprint arXiv:2310.08278, 2023.

Adam Santoro, Felix Hill, David Barrett, Ari Morcos, and Timothy Lillicrap. Measuring abstract reasoning in neural networks. In ICML, 2018.

Noam Shazeer et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In ICLR, 2017.

Mingtian Tan, Mike Merrill, Vinay Gupta, Tim Althoff, and Thomas Hartvigsen. Are language models actually useful for time series forecasting? In Advances in Neural Information Processing Systems (NeurIPS), 2024.

Qwen Team. Qwen2.5-1m: Deploying a long-context llm at scale. In arXiv preprint, 2025.

Tian Zhou, Peisong Niu, Xue Wang, Liang Sun, and Rong Jin. One fits all: Power general time series analysis by pretrained lm. Advances in Neural Information Processing Systems, 2023.

A Additional analyses

A.1 Pre- vs. post-fine-tuning representation

Loading a LoRA adapter from E1 (fine-tuned on ETTm1 forecasting) onto the frozen 8B backbone and re-probing the same synthetic windows shows that forecasting fine-tuning does not erode recognition: probe accuracy rises from 0.967 (frozen) to 0.983 (post-FT), with the largest gains on local (0.95→0.98) and mixture (0.92→0.98) windows. The representation itself shifts substantially (mean abs delta 0.69; linear CKA 0.45; probe-direction cosine 0.08), i.e., fine-tuning rewrites the recognition geometry while preserving—even sharpening—the underlying separability.

A.2 Difficulty sweeps (context, noise, horizon)

See Table 6 in the main text. Recognition degrades only mildly under noise (0.98 at σ=0.05 to 0.91 at σ=0.2) and improves with horizon, while the generation/oracle MSE ratio stays 6–9× and grows with horizon (6.5× at H=8 to 8.4× at H=32).

A.3 Tokenizer / template ablations

Prior work in this project (summarized in Appendix A.5) showed that when a random-BPE tokenizer is applied to numeric text, the model collapses to degenerate outputs, whereas fixed-width two-decimal numeric tokens (our protocol) keep generation parseable; the recognition probe is insensitive to this choice. This isolates the tokenization contribution to the generation failure channel.

A.4 Full real-data table (8B)

See Table 11.

A.5 Random-BPE tokenizer ablation

We trained a BPE tokenizer on shuffled/random text and applied it to the same numeric forecasting protocol. Under this tokenizer, frozen generation collapses (parse rate near zero), while the recognition probe is unaffected. This confirms that the generation failure channel includes a tokenization component—absent in our main protocol—and that recognition is robust to it.

A.6 Statistical details

Bootstrap 95% CIs (2000 resamples) for the headline numbers: 8B router accuracy 0.678 [0.617, 0.783] vs. random 0.289 [0.267, 0.308] vs. features 0.128 [0.108, 0.142]; generation/oracle MSE ratio 5.5× [4.0, 6.5]. The 0.6B model was extended to five seeds

Dataset	Feature MSE	Frozen-LLM MSE	Best single
Electricity	**0.740**	0.927	local/0.855
ETTh1	0.581	**0.047**	periodic/0.045
ETTh2	0.610	**0.218**	periodic/0.211
ETTm1	0.521	**0.022**	periodic/0.015
ETTm2	0.327	0.151	periodic/0.064
Exchange	1.591	**0.027**	periodic/0.027
Illness	4.620	**1.455**	trend/1.067
Traffic	**0.771**	0.916	periodic/0.883
Weather	0.482	**0.082**	periodic/0.025
M4-Daily	**1.244**	1.826	local/1.213
M4-Hourly	214.8	**214.0**	periodic/214.0
M4-Weekly	**1.419**	2.524	local/1.419
M4-Monthly	**1.218**	2.140	local/1.218
M4-Quarterly	**1.233**	1.996	local/1.233
M4-Yearly	**1.164**	1.938	local/1.164

Table 11: R6 full: 8B zero-shot routing MSE across 15 datasets.

(7/17/27/37/47): recognition 0.984 ± 0.006, generation ratio $9.9\times$, and router accuracy 0.337 [0.282, 0.395] vs. random 0.217 [0.200, 0.235] and features 0.162 [0.132, 0.192], confirming the ordering at a second scale.